

UNIVERSIDAD CARLOS III



Sistemas Distribuidos

Ejercicio Evaluable 2: Sockets TCP

CURSO 2025 - 2026

XIYA YE → 100522159@alumnos.uc3m.es

Grupo 85, Ingeniería Informática

WENJIE CHEN → 100522255@alumnos.uc3m.es

Grupo 85, Ingeniería Informática

Diseño Realizado

Para hacer un modelo cliente-servidor distribuido utilizando sockets TCP/IP, hemos empezado por aislar la aplicación cliente de la complejidad de la red. Para ello, hemos encapsulado las comunicaciones en una biblioteca dinámica (*libproxyclaves.so*) que actúa como proxy. Este proxy localiza al servidor evaluando dinámicamente las variables de entorno *IP_TUPLAS* y *PORT_TUPLAS* en cada petición. Al hacerlo así, logramos un diseño flexible, ya que el destino actualiza sin necesidad de reiniciar el cliente, al mismo tiempo que filtramos errores localmente (como los límites de caracteres o el rango de los vectores).

En el lado del servidor, hemos diseñado la aplicación (*servidor-sock.c*) para ser totalmente concurrente. El proceso principal se mantiene en un bucle de escucha activa y, ante cada nueva conexión entrante, delega el procesamiento en un hilo independiente mediante *pthread_create*. Para asegurar la integridad de los datos en este entorno multicliente, hemos implementado un mecanismo de exclusión mutua con *mutex*. Donde cada hilo debe adquirir un cerrojo global justo antes de invocar la lógica de persistencia y liberarlo inmediatamente después, lo que nos garantiza que las operaciones sobre el almacén sean atómicas y evitando así condiciones de carrera.

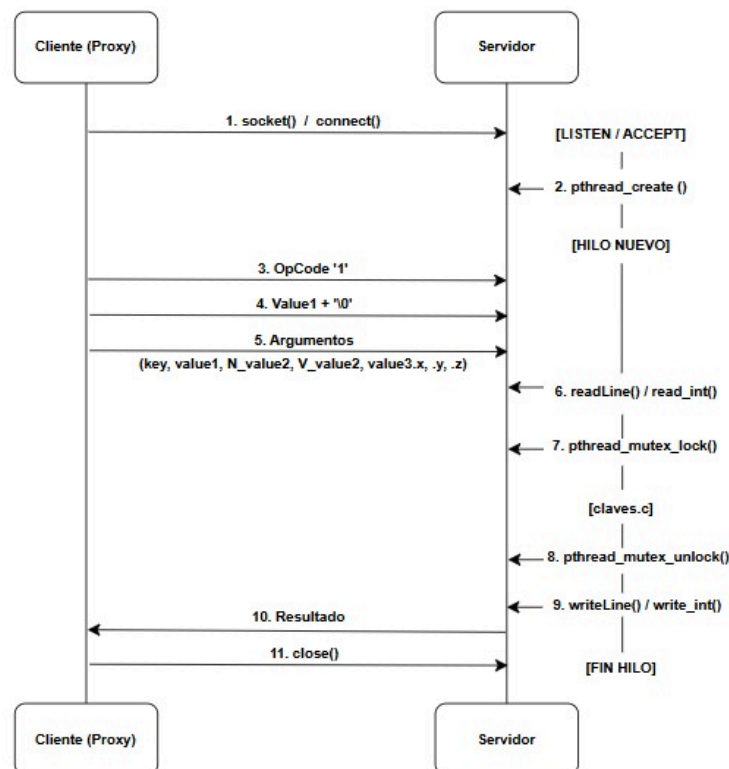
Además de la concurrencia en el servidor, hemos identificado que hay una función (*gethostbyname*) utilizada en el proxy donde nos ha dado problemas en algún ordenador al no ser segura para hilos por defecto en sistemas Linux. Por lo que para evitar condiciones de carrera al usar múltiples hilos donde el cliente intenta traducir la IP en el mismo momento, hemos implementado un mecanismo de exclusión mutua con *mutex* dentro de la biblioteca del proxy. Esto nos permite ejecutar el código en paralelo.

Respecto al almacenamiento y la interfaz de la API, hemos decidido mantener intacto el diseño del Ejercicio Evaluable 1. Por lo que se sigue utilizando el sistema basado en ficheros individuales dentro del directorio *./ALMACEN*, donde cada archivo toma el nombre de su clave. De la misma manera, el fichero de cabecera *claves.h* y la lógica de gestión de ficheros en *claves.c* no han sufrido cambios.

Diseño del Protocolo

El protocolo de aplicación se basa en un modelo de **petición/respuesta síncrono** sobre TCP, utilizando conexiones efímeras donde el cliente abre un socket para cada operación y lo cierra tras recibir la respuesta. La comunicación comienza con el envío de un código de operación (OpCode) de un byte, seguido de los argumentos de la función serializados individualmente. Para garantizar la independencia de la arquitectura, los datos numéricos y estructuras como el **Paquete** se convierten a formato de texto antes del envío, utilizando delimitadores como el carácter nulo (`\0`) para las cadenas de texto, lo que permite una reconstrucción precisa de los datos en el servidor mediante funciones de lectura segura.

En el lado del servidor, cada conexión es gestionada de forma **concurrente** mediante la creación de un hilo independiente con `pthread_create()`. El flujo de datos en el servidor incluye una fase crítica de sincronización: antes de invocar la lógica de persistencia de `claves.c` para operar sobre el directorio `./ALMACEN`, el hilo adquiere un *mutex* global. Este mecanismo de exclusión mutua garantiza que el acceso al sistema de ficheros sea atómico, evitando condiciones de carrera entre clientes simultáneos. Una vez procesada la solicitud y liberado el *mutex*, el servidor devuelve el código de resultado al proxy, completando así el ciclo de la transacción distribuida.



Compilación y Ejecución

El proceso de construcción del sistema se ha automatizado mediante un archivo *Makefile*, lo que nos garantiza que todas las dependencias entre los módulos de red, la lógica de persistencia y las aplicaciones cliente se resuelvan de manera correcta y eficiente.

Proceso de Compilación

La compilación se realiza utilizando el compilador gcc con los flags `-Wall -Wextra`, asegurando un código robusto y libre de avisos. El diseño se divide en varias etapas:

- **Bibliotecas Compartidas:** se generan dos objetivos compartidos fundamentales: *libclaves.so*, que contiene la lógica de gestión de ficheros en el servidor, y *libproxyclaves.so*, que encapsula el código de comunicaciones (proxy) para el cliente. El uso de la opción `-fPIC` permite que estas bibliotecas sean reubicables en memoria.
- **Módulos Auxiliares:** se compila de forma independiente el módulo *lines.o*, encargado de la gestión de lectura/escritura segura en sockets para evitar fragmentación de mensajes.
- **Binarios Finales:** el *Makefile* enlaza los ejecutables *servidor-sock*, *app-cliente-sock* y *app-cliente-sock-concurrente* utilizando las librerías compartidas generadas. Además, en el caso del servidor y del cliente concurrente, hemos incluido el flag `-lpthread` para la gestión de hilos POSIX.

Procedimiento de Ejecución

Para poder ejecutar el programa, debemos previamente indicar al enlazador dinámico del sistema operativo dónde puede encontrar las bibliotecas dinámicas. Esto lo debemos hacer en todas las terminales que vayan a ejecutar los binarios, con el siguiente comando:

```
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:.
```

Con esto, podemos empezar a ejecutar el programa. Se divide en dos fases:

1. Inicializamos el Servidor en la terminal 1: `./servidor-sock <PUERTO>`
2. Configuramos y ejecutamos los clientes en la terminal 2

Podemos hacerlo de dos maneras diferentes:

- a. Definiendo variables de entorno con *export*:

```
export IP_TUPLAS=<IP_SERVIDOR>  
export PORT_TUPLAS=<PUERTO>  
./app-cliente-sock  
./app-cliente-sock-concurrente
```

- b. Pasando los valores de las variables de entorno con *env*:
- ```
env IP_TUPLAS=<IP_SERVIDOR> PORT_TUPLAS=<PUERTO>
./app-cliente-sock
env IP_TUPLAS=<IP_SERVIDOR> PORT_TUPLAS=<PUERTO>
./app-cliente-sock-concurrente
```

## Batería de Pruebas

Para validar el funcionamiento de la nueva arquitectura basada en sockets TCP, hemos estructurado las pruebas en dos niveles: funcionalidad y resistencia ante concurrencia. En primer lugar, hemos decidido mantener la batería de 15 pruebas desarrollada para el Ejercicio Evaluable 1 (*app-cliente-1.c*). A continuación, hemos incluido una nueva batería de pruebas concurrentes (*app-cliente-2.c*). En estas pruebas, instanciamos 6 hilos simultáneos que actúan como clientes independientes. Donde cada hilo ejecuta un bucle de 50 iteraciones, realizando operaciones de inserción, consulta y modificación sobre el servidor.

Para que esta prueba sea efectiva, cada hilo trabaja sobre una clave única generada a partir de su identificador propio, evitando así colisiones. El éxito de estas operaciones sin pérdida de datos nos confirma que el mecanismo de Exclusión Mutua (*mutex*) que hemos implementado en el servidor protege correctamente las secciones críticas, garantizado que el acceso al directorio *./ALMACEN* sea atómico y seguro.